Student ID :	24CS047	Student Name:	Manav Lakhani			
Subject code & Name :	ITUE301: Advance Web Development Framework	Practical :	4	Academic Year	:	2026-27

Task - 1 (Practical - 4)

Aim

To design and implement a RESTful backend server with complete CRUD endpoints using Express middleware pipeline, request logging, and centralized error handling.

Code

1. server.js (Complete Express Server, Middlewares, and CRUD Routes):

```
/**
 * Charotar University of Science and Technology (CHARUSAT)
 * CSPIT - Computer Science & Engineering
 * Course: Advanced Web Development Frameworks (ITUE301)
 * Practical 4: Building a RESTful API with Node.js and Express
 * Student: Manav Lakhani (24CS047)
 */

const express = require('express');
const app = express();
const PORT = process.env.PORT || 5000;

// ========================================
// 1. Built-in JSON Body Parsing Middleware
// ========================================
app.use(express.json());

// ========================================
// 2. Global Request Logging Middleware
// Logs HTTP Method, Request URL, and Timestamp
// ========================================
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.url}`);
  next();
});

// ========================================
// 3. Supplementary Problem: Content-Type Validation Middleware
// Rejects POST / PUT requests missing 'application/json' Content-Type
// ========================================
const requireJsonContent = (req, res, next) => {
  if (['POST', 'PUT'].includes(req.method)) {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.toLowerCase().includes('application/json')) {
      return res.status(400).json({
        error: 'Bad Request',
```

```
        message: 'Requests with payload (POST/PUT) must include "Content-Type: application/json"
header.'
      });
    }
  }
  next();
};

app.use(requireJsonContent);

// ========================================
// 4. In-Memory Task Storage (Temporary Array)
// ========================================
let tasks = [
  {
    id: 1,
    title: 'Complete Practical 4 Lab',
    description: 'Implement Express RESTful API with full CRUD and middleware pipeline.',
    status: 'in-progress',
    createdAt: new Date('2026-10-11T10:00:00.000Z').toISOString(),
    updatedAt: new Date('2026-10-11T10:00:00.000Z').toISOString()
  },
  {
    id: 2,
    title: 'Review IBM Coursera Module 1',
    description: 'Study REST API design, middleware lifecycle, and error handling.',
    status: 'completed',
    createdAt: new Date('2026-10-11T11:00:00.000Z').toISOString(),
    updatedAt: new Date('2026-10-11T11:00:00.000Z').toISOString()
  },
  {
    id: 3,
    title: 'Prepare Postman Test Collection',
    description: 'Verify status codes: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500
Error.',
    status: 'pending',
    createdAt: new Date('2026-10-11T12:00:00.000Z').toISOString(),
    updatedAt: new Date('2026-10-11T12:00:00.000Z').toISOString()
  }
];

let nextId = 4;

// ========================================
// 5. Supplementary Problem: Route-Specific ID Validation Middleware
// Validates that :id is a positive integer before reaching controllers
// ========================================
const validateTaskId = (req, res, next) => {
  const parsedId = Number(req.params.id);
  if (!Number.isInteger(parsedId) || parsedId <= 0) {
    return res.status(400).json({
      error: 'Invalid Task ID',
      message: `The provided ID '${req.params.id}' is invalid. Task ID must be a positive
integer.`
    });
```

```
  }
  req.validatedId = parsedId;
  next();
};

// ========================================
// 6. RESTful CRUD Routes for Tasks
// ========================================

// GET / - Root Health / Welcome endpoint
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'Welcome to Task Manager RESTful API (Practical 4)',
    author: 'Manav Lakhani (24CS047)',
    course: 'ITUE301: Advanced Web Development Frameworks',
    version: '1.0.0',
    endpoints: {
      getAllTasks: 'GET /tasks',
      getTaskById: 'GET /tasks/:id',
      createTask: 'POST /tasks',
      updateTask: 'PUT /tasks/:id',
      deleteTask: 'DELETE /tasks/:id'
    }
  });
});

// GET /tasks - Read all tasks (with optional query filter by status)
app.get('/tasks', (req, res) => {
  const { status, search } = req.query;
  let result = tasks;

  if (status) {
    result = result.filter(t => t.status.toLowerCase() === status.toLowerCase());
  }
  if (search) {
    result = result.filter(t =>
      t.title.toLowerCase().includes(search.toLowerCase()) ||
      t.description.toLowerCase().includes(search.toLowerCase())
    );
  }

  res.status(200).json({
    success: true,
    count: result.length,
    data: result
  });
});

// GET /tasks/:id - Read single task by ID (uses validateTaskId middleware)
app.get('/tasks/:id', validateTaskId, (req, res) => {
  const task = tasks.find(t => t.id === req.validatedId);
  if (!task) {
    return res.status(404).json({
      error: 'Not Found',
      message: `Task with ID ${req.validatedId} not found.`
```

```
    });
  }

  res.status(200).json({
    success: true,
    data: task
  });
});

// POST /tasks - Create a new task (Returns 201 Created)
app.post('/tasks', (req, res) => {
  const { title, description, status } = req.body;

  // Validation: Title is required
  if (!title || typeof title !== 'string' || title.trim() === '') {
    return res.status(400).json({
      error: 'Validation Error',
      message: 'Task title is required and cannot be empty.'
    });
  }

  const validStatuses = ['pending', 'in-progress', 'completed'];
  const taskStatus = status && validStatuses.includes(status.toLowerCase())
    ? status.toLowerCase()
    : 'pending';

  const now = new Date().toISOString();
  const newTask = {
    id: nextId++,
    title: title.trim(),
    description: description ? description.trim() : '',
    status: taskStatus,
    createdAt: now,
    updatedAt: now
  };

  tasks.push(newTask);

  res.status(201).json({
    success: true,
    message: 'Task created successfully.',
    data: newTask
  });
});

// PUT /tasks/:id - Update an existing task (uses validateTaskId middleware)
app.put('/tasks/:id', validateTaskId, (req, res) => {
  const taskIndex = tasks.findIndex(t => t.id === req.validatedId);
  if (taskIndex === -1) {
    return res.status(404).json({
      error: 'Not Found',
      message: `Task with ID ${req.validatedId} does not exist.`
    });
  }
```

```
  const { title, description, status } = req.body;

  // Partial or full update
  if (title !== undefined) {
    if (typeof title !== 'string' || title.trim() === '') {
      return res.status(400).json({
        error: 'Validation Error',
        message: 'Title must be a non-empty string.'
      });
    }
    tasks[taskIndex].title = title.trim();
  }

  if (description !== undefined) {
    tasks[taskIndex].description = typeof description === 'string' ? description.trim() : '';
  }

  if (status !== undefined) {
    const validStatuses = ['pending', 'in-progress', 'completed'];
    if (!validStatuses.includes(status.toLowerCase())) {
      return res.status(400).json({
        error: 'Validation Error',
        message: `Status must be one of: ${validStatuses.join(', ')}.`
      });
    }
    tasks[taskIndex].status = status.toLowerCase();
  }

  tasks[taskIndex].updatedAt = new Date().toISOString();

  res.status(200).json({
    success: true,
    message: `Task with ID ${req.validatedId} updated successfully.`,
    data: tasks[taskIndex]
  });
});

// DELETE /tasks/:id - Delete a task by ID (uses validateTaskId middleware)
app.delete('/tasks/:id', validateTaskId, (req, res) => {
  const taskIndex = tasks.findIndex(t => t.id === req.validatedId);
  if (taskIndex === -1) {
    return res.status(404).json({
      error: 'Not Found',
      message: `Task with ID ${req.validatedId} not found.`
    });
  }

  const [deletedTask] = tasks.splice(taskIndex, 1);

  res.status(200).json({
    success: true,
    message: `Task with ID ${req.validatedId} deleted successfully.`,
    data: deletedTask
  });
});
```

```
// ========================================
// 7. Faculty Demonstration Route: Deliberate Error Trigger
// Demonstrates global error handler catching unhandled exceptions
// ========================================
app.get('/trigger-error', (req, res, next) => {
  const err = new Error('Demonstration: Deliberately triggered internal server error.');
  err.status = 500;
  next(err);
});

// ========================================
// 8. Supplementary Problem: Custom 404 Handler for Undefined Routes
// Catches any requests that did not match registered routes
// ========================================
app.use((req, res) => {
  res.status(404).json({
    error: 'Route Not Found',
    message: `Cannot ${req.method} ${req.originalUrl}. This endpoint does not exist.`,
    validRoutes: [
      'GET    /tasks',
      'GET    /tasks/:id',
      'POST   /tasks',
      'PUT    /tasks/:id',
      'DELETE /tasks/:id',
      'GET    /trigger-error'
    ]
  });
});

// ========================================
// 9. Global Centralized Error Handling Middleware
// Defined LAST in the middleware pipeline with (err, req, res, next)
// ========================================
app.use((err, req, res, next) => {
  console.error('[Global Error Handler] Caught Exception:\n', err.stack || err.message);

  const statusCode = err.status || 500;
  res.status(statusCode).json({
    error: err.name || 'Internal Server Error',
    message: err.message || 'Something went wrong on the server.'
  });
});

// ========================================
// 10. Start HTTP Server
// ========================================
if (require.main === module) {
  app.listen(PORT, () => {
    console.log(`=====================================================`);
    console.log(`Task Manager REST API Server is running on port ${PORT}`);
    console.log(`Base URL: http://localhost:${PORT}`);
    console.log(`Logging Middleware: Active`);
    console.log(`Global Error Handler: Active`);
    console.log(`=====================================================`);
```

```
  });
}

module.exports = app;
```

2. test-api.js (Automated REST API Test Suite):

```
const app = require('./server');
const http = require('http');

let server;
const PORT = 5055;
const BASE_URL = `http://localhost:${PORT}`;

async function request(path, options = {}) {
  const url = `${BASE_URL}${path}`;
  const response = await fetch(url, options);
  let data;
  try {
    data = await response.json();
  } catch (e) {
    data = await response.text();
  }
  return {
    status: response.status,
    headers: Object.fromEntries(response.headers.entries()),
    data
  };
}

async function runTests() {
  console.log('============================================================');
  console.log('  Practical 4: Express RESTful API Automated Verification Test ');
  console.log('  Student: Manav Lakhani (24CS047) | ITUE301                    ');
  console.log('============================================================\n');

  let passed = 0;
  let total = 0;

  async function testCase(name, fn) {
    total++;
    try {
      await fn();
      console.log(`[PASS] Test ${total}: ${name}`);
      passed++;
    } catch (err) {
      console.error(`[FAIL] Test ${total}: ${name}`);
      console.error(`       Error: ${err.message}`);
    }
  }

  // 1. Health check
  await testCase('GET / - API root returns 200 OK', async () => {
    const res = await request('/');
    if (res.status !== 200) throw new Error(`Expected 200, got ${res.status}`);
```

```
    if (!res.data.endpoints) throw new Error('Endpoints overview missing');
  });

  // 2. GET /tasks
  await testCase('GET /tasks - Retrieve all tasks returns 200 OK with array', async () => {
    const res = await request('/tasks');
    if (res.status !== 200) throw new Error(`Expected 200, got ${res.status}`);
    if (!Array.isArray(res.data.data)) throw new Error('Data is not an array');
    if (res.data.count < 1) throw new Error('Expected at least 1 seeded task');
  });

  // 3. POST /tasks without Content-Type (Supplementary Problem 1)
  await testCase('POST /tasks without Content-Type header rejected with 400 Bad Request', async ()
=> {
    const res = await request('/tasks', {
      method: 'POST',
      body: JSON.stringify({ title: 'Invalid Content-Type Task' })
      // Notice: No headers provided!
    });
    if (res.status !== 400) throw new Error(`Expected 400, got ${res.status}`);
    if (res.data.error !== 'Bad Request') throw new Error(`Unexpected error message:
${JSON.stringify(res.data)}`);
  });

  // 4. POST /tasks with valid payload
  let createdTaskId;
  await testCase('POST /tasks with valid payload returns 201 Created and new task', async () => {
    const payload = {
      title: 'Automated Test Task',
      description: 'Created during test suite execution',
      status: 'pending'
    };
    const res = await request('/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });
    if (res.status !== 201) throw new Error(`Expected 201, got ${res.status}`);
    if (res.data.data.title !== payload.title) throw new Error('Title mismatch');
    createdTaskId = res.data.data.id;
  });

  // 5. POST /tasks with missing title
  await testCase('POST /tasks with missing title returns 400 Validation Error', async () => {
    const res = await request('/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ description: 'No title' })
    });
    if (res.status !== 400) throw new Error(`Expected 400, got ${res.status}`);
    if (res.data.error !== 'Validation Error') throw new Error('Expected Validation Error');
  });

  // 6. GET /tasks/:id (valid ID)
  await testCase('GET /tasks/:id with valid existing ID returns 200 OK', async () => {
```

```
    const res = await request(`/tasks/${createdTaskId}`);
    if (res.status !== 200) throw new Error(`Expected 200, got ${res.status}`);
    if (res.data.data.id !== createdTaskId) throw new Error('Task ID mismatch');
  });

  // 7. GET /tasks/:id with non-numeric ID (Supplementary Problem 2)
  await testCase('GET /tasks/:id with invalid ID format ("abc") rejected with 400 Bad Request',
async () => {
    const res = await request('/tasks/abc');
    if (res.status !== 400) throw new Error(`Expected 400, got ${res.status}`);
    if (res.data.error !== 'Invalid Task ID') throw new Error('Expected Invalid Task ID');
  });

  // 8. GET /tasks/:id with non-existent numeric ID
  await testCase('GET /tasks/9999 with non-existent ID returns 404 Not Found', async () => {
    const res = await request('/tasks/9999');
    if (res.status !== 404) throw new Error(`Expected 404, got ${res.status}`);
  });

  // 9. PUT /tasks/:id
  await testCase('PUT /tasks/:id with valid updates returns 200 OK and updated task', async () =>
{
    const res = await request(`/tasks/${createdTaskId}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ status: 'completed', description: 'Updated successfully' })
    });
    if (res.status !== 200) throw new Error(`Expected 200, got ${res.status}`);
    if (res.data.data.status !== 'completed') throw new Error('Status not updated');
  });

  // 10. DELETE /tasks/:id
  await testCase('DELETE /tasks/:id removes task and returns 200 OK', async () => {
    const res = await request(`/tasks/${createdTaskId}`, {
      method: 'DELETE'
    });
    if (res.status !== 200) throw new Error(`Expected 200, got ${res.status}`);

    // Verify task is now deleted
    const verifyRes = await request(`/tasks/${createdTaskId}`);
    if (verifyRes.status !== 404) throw new Error('Deleted task still accessible');
  });

  // 11. Undefined route custom 404 handler (Supplementary Problem 3)
  await testCase('GET /api/undefined-route returns structured 404 JSON response', async () => {
    const res = await request('/api/undefined-route');
    if (res.status !== 404) throw new Error(`Expected 404, got ${res.status}`);
    if (res.data.error !== 'Route Not Found') throw new Error('Expected Route Not Found');
    if (!Array.isArray(res.data.validRoutes)) throw new Error('Expected validRoutes list');
  });

  // 12. Deliberate unhandled error caught by Global Error Handler
  await testCase('GET /trigger-error triggers centralized 500 error handler', async () => {
    const res = await request('/trigger-error');
    if (res.status !== 500) throw new Error(`Expected 500, got ${res.status}`);
```

```
    if (res.data.error !== 'Error') throw new Error(`Unexpected error format:
${JSON.stringify(res.data)}`);
  });

  console.log('\n============================================================');
  console.log(`  Test Results Summary: ${passed} / ${total} Tests Passed
(${Math.round((passed/total)*100)}%)`);
  console.log('============================================================\n');

  server.close();
}

server = app.listen(PORT, () => {
  runTests();
});
```

3. package.json:

```
{
  "name": "task-manager-api-24cs047",
  "version": "1.0.0",
  "description": "RESTful API with Node.js and Express for Practical 4 (ITUE301)",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js",
    "test": "node test-api.js"
  },
  "keywords": [
    "express",
    "rest-api",
    "crud",
    "middleware",
    "task-manager"
  ],
  "author": "Manav Lakhani (24CS047)",
  "license": "ISC",
  "dependencies": {
    "express": "^4.22.3"
  }
}
```

GitHub Repository Link: https://github.com/24CS047Manav/Awdf_Practica_4

Output

1. Server Initialization & Middleware Pipeline:
Configured an Express server listening on port 5000 with express.json() parser, global logging, route controllers, 404 handler, and global error middleware.

2. Global Request Logging Middleware (Step 3):
Every incoming HTTP request triggers the logging middleware which outputs the method, URL path, and ISO timestamp (e.g. "[2026-10-11T18:23:59.708Z] GET /tasks").

3. Read All Tasks (GET /tasks - 200 OK):
Returns the array of all task objects with HTTP status code 200 OK. Supports optional query parameters ?status= and ?search= for filtering.

4. Create Task (POST /tasks - 201 Created):
Receives JSON payload, generates a unique ID, records creation/update timestamps, and appends the new task to the in-memory array returning HTTP 201 Created.

5. Read Single Task by ID (GET /tasks/:id - 200 OK / 404 Not Found):
Finds task by identifier. If present, returns HTTP 200 OK. If not found, returns HTTP 404 with a structured error response.

6. Update Task (PUT /tasks/:id - 200 OK / 404 Not Found):
Modifies task title, description, or status, updates the updatedAt timestamp, and returns the modified object with HTTP 200 OK.

7. Delete Task (DELETE /tasks/:id - 200 OK / 404 Not Found):
Removes the specified task from the storage array and returns confirmation with HTTP 200 OK.

8. Supplementary Problem 1 - Content-Type Header Enforcement:
Requests to POST /tasks and PUT /tasks without the "Content-Type: application/json" header are intercepted and rejected with HTTP 400 Bad Request.

9. Supplementary Problem 2 - Route-Specific Task ID Validation:
A specialized middleware validateTaskId intercepts routes with :id parameter, rejecting non-numeric or non-positive IDs (e.g., /tasks/abc) with HTTP 400 Bad Request before hitting controller logic.

10. Supplementary Problem 3 - Structured 404 Handler for Undefined Routes:
Any request sent to an unmapped path (e.g., GET /api/undefined-route) returns a clean JSON error response listing all valid available endpoints with HTTP 404 Not Found.

11. Centralized Global Error Handler (Step 5):
A 4-argument error handling middleware (err, req, res, next) declared as the final middleware catches unhandled exceptions (verified via GET /trigger-error) returning HTTP 500 without leaking raw internal stack traces to clients.

12. Automated Verification & Test Results (100% Pass):
Executed automated verification script (test-api.js) executing all 12 test assertions covering all CRUD operations, error scenarios, and status codes (200, 201, 400, 404, 500) with 100% pass rate.

Test Suite Execution Output Log:

```
============================================================
  Practical 4: Express RESTful API Automated Verification Test
```

```
  Student: Manav Lakhani (24CS047) | ITUE301
============================================================

[2026-10-11T18:23:59.687Z] GET /
[PASS] Test 1: GET / - API root returns 200 OK
[2026-10-11T18:23:59.708Z] GET /tasks
[PASS] Test 2: GET /tasks - Retrieve all tasks returns 200 OK with array
[2026-10-11T18:23:59.729Z] POST /tasks
[PASS] Test 3: POST /tasks without Content-Type header rejected with 400 Bad Request
[2026-10-11T18:23:59.747Z] POST /tasks
[PASS] Test 4: POST /tasks with valid payload returns 201 Created and new task
[2026-10-11T18:23:59.752Z] POST /tasks
[PASS] Test 5: POST /tasks with missing title returns 400 Validation Error
[2026-10-11T18:23:59.755Z] GET /tasks/4
[PASS] Test 6: GET /tasks/:id with valid existing ID returns 200 OK
[2026-10-11T18:23:59.760Z] GET /tasks/abc
[PASS] Test 7: GET /tasks/:id with invalid ID format ("abc") rejected with 400 Bad Request
[2026-10-11T18:23:59.763Z] GET /tasks/9999
[PASS] Test 8: GET /tasks/9999 with non-existent ID returns 404 Not Found
[2026-10-11T18:23:59.767Z] PUT /tasks/4
[PASS] Test 9: PUT /tasks/:id with valid updates returns 200 OK and updated task
[2026-10-11T18:23:59.774Z] DELETE /tasks/4
[2026-10-11T18:23:59.778Z] GET /tasks/4
[PASS] Test 10: DELETE /tasks/:id removes task and returns 200 OK
[2026-10-11T18:23:59.781Z] GET /api/undefined-route
[PASS] Test 11: GET /api/undefined-route returns structured 404 JSON response
[2026-10-11T18:23:59.785Z] GET /trigger-error
[Global Error Handler] Caught Exception:
 Error: Demonstration: Deliberately triggered internal server error.
[PASS] Test 12: GET /trigger-error triggers centralized 500 error handler

============================================================
  Test Results Summary: 12 / 12 Tests Passed (100%)
============================================================
```